

Model Evaluation

Speaker notes for all 35 slides

How to use these notes

Each section is labeled with the corresponding slide number and title. The script is written to sound natural when spoken. A presenter cue appears only when a slide benefits from an extra clarification or a caution about interpretation.

A quick accuracy reminder

Keep the final test set separate from model selection. Use training data to fit, validation data or cross-validation to choose, and the untouched test data for the final report.

SLIDE-BY-SLIDE SCRIPT

SLIDE 01 OF 35

Model evaluation

Today we begin with a practical question: a classifier can assign labels, but how do we know whether it is useful? We will look at how to evaluate its predictions on data it did not use to learn, and how to describe the kinds of errors it makes.

SLIDE 02 OF 35

Six steps: how to evaluate classification

Here is our route. We will split the data, use cross-validation to make evaluation less dependent on one split, read a confusion matrix, and calculate several metrics. Then we will see how the classification threshold changes the decisions and finish with ROC curves and AUC.

SLIDE 03 OF 35

We can classify. Is the classifier useful?

Last time, we studied classifiers such as logistic regression, LDA, QDA, and KNN. They can make different predictions on the same problem. Today we are asking a different question: how well do those predictions work, and what kinds of errors do they make?

Three methods, one scorecard

These methods produce predictions in different ways. Logistic regression estimates a probability and applies a cutoff. LDA and QDA compare how well a case fits each class distribution. KNN looks at nearby training examples and uses their labels. Since the methods work differently, we need a consistent way to evaluate them.

Train and test

A model learns patterns from training data. To evaluate it fairly, we need examples that were not used to fit it. We will train on one set and grade the model on another set that stands in for future data.

Hold something out

In the first setup, we use 70 percent of the rows for training and hold 30 percent out for testing. In the second setup, we also use a validation set: training data fits the model, validation data helps us choose settings, and the test set is saved for the final evaluation. We should finish our choices before looking at test results.

PRESENTER CUE

Keep the test set out of model selection. If we repeatedly use test results to choose a model or cutoff, it is no longer an independent final check.

Three settings that matter

Here, `test_size=0.25` means 25 percent of the rows are held out for testing. `random_state=0` makes the shuffle reproducible; zero is just a seed, not a special statistical value. For a classifier, score on the test set commonly reports accuracy: the share of test cases classified correctly.

One split can look lucky

The picture compares two possible test splits: the model gets 9 of 10 correct on one and 7 of 10 on the other. With only ten test cases, a couple of different examples can noticeably change the score. The modeling approach and source dataset are the same, but in a proper split-based evaluation we refit the model on each training portion and test it on its held-out portion.

PRESENTER CUE

Say "same model type or procedure" rather than implying the exact same fitted model was tested on two independently drawn samples.

Cross-validation

Cross-validation repeats this train-and-test idea across several folds. We divide the available data into folds, hold one fold out for testing, and train on the others. Then we rotate the held-out fold so each row gets a turn as test data.

Split, rotate, average

First, divide the data into k roughly equal folds. For one round, train on k minus 1 folds and test on the remaining fold. Repeat until each fold has been the test fold once. Then summarize the k test scores, often by taking their average. With five folds, each round trains on about 80 percent and tests on about 20 percent.

Five folds, five tests

Each row in this diagram is one round of cross-validation. The orange fold is held out for testing in that round; the blue folds are used for training. In the next round, a different fold becomes orange. By the end, every fold has been tested once.

A single split hides the range

These bars show five accuracy scores from five held-out folds for logistic regression on the Iris data. They range from 0.90 to 1.00, and their average is 0.96. One split would show only one of these results, while cross-validation lets us see how performance varied across folds.

PRESENTER CUE

The chart scale begins at 0.85, not zero, to make small differences visible. Tell students that explicitly; otherwise the differences may look larger than they are. The y-axis should be labeled Accuracy, and the stray text "what is y" should be removed.

Keep the class mix

Ordinary folds can cause trouble if the data are ordered by class and we take consecutive chunks: a test fold might contain only one class, while the corresponding training data may not include that class. Stratified folds preserve approximately the same class proportions in every fold as in the full dataset. For classification, stratification is a common default.

PRESENTER CUE

This example assumes rows are class-sorted and folds are made without shuffling. Plain K-fold does not always fail; the risk depends on the row order and splitting method.

Three jobs for the data

Training data are used to fit the model. Validation data help us choose among models or settings, such as K or a threshold. The test set is held back for a final report. Cross-validation can provide the rotating validation folds, but a separate test set should remain untouched until model choices are complete.

PRESENTER CUE

If you are using cross-validation to tune settings, apply it within the training data. Do not use the final test set as one of the tuning folds.

The confusion matrix

Now we will examine the confusion matrix. It compares the actual class with the predicted class. For a binary classifier, it organizes the results into four outcomes: true negatives, false positives, false negatives, and true positives.

1,000 loan applicants

In this example, the positive class is an applicant who will default. The rows show what actually happened, and the columns show the model prediction. We have 900 true negatives, 50 false positives, 20 false negatives, and 30 true positives. Let us first read what each count means.

What those four numbers say

Of the 950 people who did not default, the model correctly labeled 900 and incorrectly flagged 50. Of the 50 people who did default, it caught 30 and missed 20. Also notice that 80 applicants were flagged as likely to default, and only 30 of those flags were correct.

A false alarm and a miss

A false positive is a false alarm: the model flags someone who does not default. A false negative is a miss: the model predicts no default, but the person does default. Those errors can have different costs. The medical screening examples on the slide illustrate the same structure: a healthy person flagged versus a sick person missed.

PRESENTER CUE

Connect each error to the loan example too: the 50 false positives are good customers flagged; the 20 false negatives are defaults the model missed.

Beyond accuracy

We can summarize a classifier in more than one way. Accuracy is one useful starting point, but it can hide important information. The next metrics answer different questions about the model and its errors.

Accuracy

Accuracy is the number of correct predictions divided by the total number of predictions. Here, 900 true negatives plus 30 true positives gives 930 correct out of 1,000, or 93 percent. But 95 percent of these applicants did not default. A model that always predicted no default would be 95 percent accurate and would catch none of the actual defaults. That is why we should compare accuracy with the class balance and the problem we care about.

PRESENTER CUE

This example is especially useful: the displayed model has 93 percent accuracy, below the 95 percent majority-class baseline. Do not imply that accuracy alone proves this model is useful.

Precision

Precision asks: among the applicants the model flagged as likely to default, what fraction actually defaulted? There were 80 flags and 30 real defaults, so precision is 30 divided by 80, or 0.375. In other words, 37.5 percent of the flags were real defaults. Precision is useful when false alarms are costly.

Recall

Recall asks: among all the people who actually defaulted, how many did the model catch? It caught 30 of 50, so recall is 0.60. That means the model found 60 percent of the defaults and missed the other 40 percent. When missing a positive case is costly, recall may be especially important.

PRESENTER CUE

Precision and recall often trade off as we change the threshold, but the exact pattern depends on the data and the model.

Specificity

Specificity asks: among people who did not default, how many did the model correctly clear? It correctly cleared 900 of 950, so specificity is about 0.947. The false positive rate is one minus specificity, about 0.053. That false positive rate will be the horizontal coordinate in our ROC curve.

The F1 score

The F1 score combines precision and recall using their harmonic mean. With precision 0.375 and recall 0.60, the F1 score is about 0.462. Because it depends on both precision and recall, one cannot be high while the other is very low and still produce a high F1. F1 leaves true negatives out, so it does not tell the whole story either.

The threshold

A classifier may produce a probability, but we still need a rule for turning that probability into a class label. That rule is the threshold, or cutoff. We will see that changing the cutoff changes the predictions and therefore changes the confusion matrix and its metrics.

A score is not yet a label

This applicant has a predicted probability of default of 0.72. At a threshold of 0.50, 0.72 is above the cutoff, so the classifier labels the applicant as default. The probability is a model estimate, not a guarantee about what this individual will do.

Recall and precision trade places

With a lower cutoff, more applicants are labeled positive. In this example, lowering it to 0.30 flags six of the eight applicants; raising it to 0.70 flags only two. A lower cutoff tends to catch more actual positives, but can add false alarms. A higher cutoff usually flags fewer people, which can reduce false alarms but may miss more actual positives.

PRESENTER CUE

The diagram shows this example-specific change. In general, recall cannot decrease when the positive threshold is lowered, but precision does not have to move monotonically for every dataset.

ROC and AUC

So far, we have mostly discussed metrics at a particular cutoff. The ROC curve gives us a view across many cutoffs, and AUC summarizes that curve. Let us start by understanding the two rates on its axes.

True positive rate against false positive rate

The vertical axis is the true positive rate, which is recall. The horizontal axis is the false positive rate, the share of actual negatives incorrectly labeled positive. Every threshold gives a point on this graph. A curve that moves toward the upper left catches more positives while creating fewer false alarms. The diagonal represents chance-level ranking.

Area under the ROC curve

AUC is the area under the ROC curve. An AUC of 1 means perfect ranking: every positive case is ranked above every negative case. An AUC around 0.5 is like chance-level ranking. A useful interpretation is that AUC measures how often a randomly chosen positive case receives a higher score than a randomly chosen negative case, with ties handled halfway.

PRESENTER CUE

AUC evaluates ranking across thresholds. It does not select the cutoff, and it does not tell us whether predicted probabilities are well calibrated.

One cutoff, and all of them

At the cutoff we plan to use, we examine the actual decisions: the confusion matrix, accuracy, precision, recall, specificity, and F1. Across cutoffs, the ROC curve shows the ranking tradeoff and AUC summarizes it. These answer related but different questions, so we should use both kinds of information.

From data to a number you can defend

Here is the full workflow. First, set aside test rows. Fit the model on training data. Use validation data or cross-validation to tune choices. Then generate class predictions and probabilities for the final test data, calculate appropriate evaluation measures, and report the metric and cutoff clearly.

PRESENTER CUE

When using cross-validation for tuning, keep the final test data outside the cross-validation process. Fit preprocessing steps within each training fold to avoid data leakage.

What the code is scoring

The code makes two kinds of output. `predict` gives class labels after applying a cutoff, so we use those labels for the confusion matrix, precision, recall, and F1. `predict_proba` gives probabilities before the cutoff; here `[:, 1]` selects the probability for class 1, default. ROC AUC uses those probabilities because it evaluates ranking across thresholds.

PRESENTER CUE

In scikit-learn, confirm that class 1 is the positive class and that the probability column matches it. Otherwise, the metrics may describe the wrong class.

Six sentences

Let us close with six reminders. Evaluate on rows not used to fit the model. Read the confusion matrix to separate hits, misses, and false alarms. Check whether class imbalance makes accuracy misleading. Choose metrics that match the question. Remember that the threshold changes the balance of errors. ROC shows behavior across thresholds, while AUC summarizes ranking.

Chapter 5, and a market to practice on

For the reading, review Chapter 5 in ISLP and Chapter 5 in the Müller and Guido text. The Smarket dataset gives us a new classification problem: predict whether the market direction is Up or Down. Next, we will use it to practice evaluation metrics.
